accessible, even if the upper layers delete them. For example, you shouldn't package passwords or encryption keys within a container image.

So far you have learned how container images are composed, but you haven't yet seen how to create one. That's coming up next.

6.1.2 Creating images with Dockerfiles

Following the OCI format, you can define a container image by listing the sequence of instructions in a particular file called a *Dockerfile*. It's a script that acts as a recipe containing all the steps to *build* the desired image.

In a Dockerfile, each instruction starts with a command from the Docker-specific syntax. Then you can pass familiar shell commands as arguments to the instructions, depending on which Linux distribution you're using as your base image. The format is as follows:

```
INSTRUCTION arguments
```

NOTE Docker supports Linux containers on machines with AMD64 and ARM64 architectures. It also supports working with Windows containers (runnable only on Windows systems), but we will exclusively use Linux containers in this book.

Let's put that into practice by defining a Dockerfile to build the container image mentioned in the previous section, composed of the following main instructions:

1. Use Ubuntu as the base image.
2. Install the Java Runtime Environment.
3. Run the `java --version` command.

Create a my-java-image folder and an empty file inside it named Dockerfile, with no extension (Chapter06/06-end/my-java-image). You might name it